



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251218
Nama Lengkap	Agatha Sabda Alethea Prabawa
Minggu ke / Materi	09 / Pengolahan String dan Regular Expression

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengantar String

String merupakan rangkaian karakter yang membentuk satu kesatuan yang biasa digunakan dalam berbagai program komputer untuk menyimpan teks, baik berupa kalimat pendek atau paragraf panjang. Dari sisi tipe data, string tergolong tipe data non-primitif karena ia menyimpan lebih dari satu nilai tunggal. Setiap karakter di dalam string direpresentasikan dalam kode ASCII, sehingga huruf, angka, ataupun simbol bisa tercakup di dalamnya.

Tidak semua bahasa pemrograman menyediakan tipe data string secara bawaan. Bahasa C misalnya, ia tidak mengenal tipe data string dan sebagai gantinya string direpresentasikan sebagai array of character. Beberapa bahasa lain pun melihat string sebagai Kumpulan karakter atau list of character. Berbeda dengan C, python menghadirkan string sebagai tipe data tersendiri yang mudah digunakan dan dilengkapi berbagai metode bawaan yang banyak fiturnya.

Pengaksesan String dan Manipulasi String

Membuat sebuah string di python cukup dengan mendeklarasikan variabel dan langsung memberikan nilainya berupa teks. Python mendukung penggunaan tanda petik tunggal maupun ganda secara bergantian, dan dua string pun dapat digabungkan menggunakan operator +.

Satu hal penting yang perlu dipahami adalah bahwa string bersifat immutable, artinya karakter di dalam string tidak dapat diubah secara langsung setelah string terbentuk. Upaya mengubah satu karakter akan menghasilkan TypeError. Untuk memodifikasi isi string, solusinya adalah membentuk string baru dari potongan-potongan string asli.

Operator dan Metode String

Operator in dan Perbandingan String

Python menyediakan operator in yang berguna untuk memeriksa apakah suatu teks merupakan bagian (substring) dari string lain. Operator ini menghasilkan nilai boolean True atau False. Selain itu, string juga dapat dibandingkan menggunakan operator relasional seperti ==, >, dan <, yang melakukan perbandingan berdasarkan urutan leksikografis (alfabet) karakter.

Fungsi len() dan Traversing String

Panjang sebuah string (jumlah karakter di dalamnya) dapat diketahui menggunakan fungsi bawaan `len()`. Karena indeks dimulai dari 0, karakter terakhir berada pada posisi `len(string) - 1`, atau cukup ditulis dengan indeks `-1`. Untuk menampilkan karakter string satu per satu, tersedia dua cara traversing menggunakan perulangan `while` dengan kontrol indeks manual, atau menggunakan perulangan `for` yang lebih ringkas dan Pythonic.

String Slice

Fitur string slice memungkinkan pengambilan sebagian karakter dari string dengan sintaks `string[awal:akhir]`. Indeks awal bersifat inklusif, sedangkan akhir bersifat eksklusif (karakter pada indeks akhir tidak ikut diambil). Salah satu atau kedua batas boleh dikosongkan, jika awal dikosongkan maka dianggap mulai dari indeks 0, dan jika akhir dikosongkan maka dianggap sampai akhir string.

Metode-Metode String yang Sering Digunakan

Python menyediakan sejumlah metode bawaan untuk memanipulasi string. Seluruh metode yang mengembalikan nilai string akan menghasilkan objek string baru, string asli tidak pernah berubah, karena sifat immutable-nya. Tabel berikut merangkum metode-metode yang paling sering dipakai:

Metode	Kegunaan
<code>capitalize()</code>	Mengubah huruf pertama menjadi huruf kapital
<code>count(sub)</code>	Menghitung berapa kali substring muncul dalam string
<code>endswith(akhiran)</code>	Mengecek apakah string berakhir dengan akhiran tertentu
<code>startswith(awalan)</code>	Mengecek apakah string dimulai dengan awalan tertentu
<code>find(sub)</code>	Mengembalikan indeks pertama kemunculan substring (-1 jika tidak ada)
<code>islower()</code> / <code>isupper()</code>	Mengecek apakah seluruh karakter huruf kecil / huruf besar
<code>isdigit()</code>	Mengecek apakah seluruh karakter merupakan angka
<code>strip()</code>	Menghapus spasi (whitespace) di awal dan akhir string

Metode	Kegunaan
<code>split(pemisah)</code>	Memecah string menjadi list berdasarkan pemisah tertentu
<code>lower()</code> / <code>upper()</code>	Mengubah seluruh karakter menjadi huruf kecil / huruf besar

Operator * dan + pada String

Selain untuk operasi aritmetika, operator + dan * pada Python memiliki peran khusus ketika diterapkan pada string. Operator + berfungsi untuk menggabungkan dua string menjadi satu, sementara operator * digunakan untuk mengulang string sebanyak kelipatan yang ditentukan.

Parsing String

Parsing string adalah proses menelusuri dan memilah-milah bagian tertentu dari sebuah teks untuk keperluan ekstraksi, transformasi, atau analisis data. Teknik ini umumnya memanfaatkan metode `split()` untuk memecah string menjadi token-token, kemudian setiap token diperiksa dan diolah lebih lanjut.

Sebagai contoh kasus, misalkan terdapat kalimat: "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka". Dari kalimat itu, kita ingin mengekstrak tanggal dan menyusunnya ulang dari format DD-MM-YYYY menjadi MM/DD/YYYY. Langkah penyelesaiannya adalah sebagai berikut: Pertama, kalimat dipecah berdasarkan spasi sehingga menghasilkan token-token: "Saudara-saudara", "pada", "tanggal", "17-08-1945", "Indonesia", dan "merdeka". Kedua, dari daftar token tersebut, dicari token yang karakter pertamanya adalah digit. Ketiga, token tanggal yang ditemukan kembali dipecah menggunakan tanda hubung - sebagai pemisah, sehingga diperoleh bagian hari, bulan, dan tahun. Keempat, ketiga bagian itu disusun ulang sesuai format yang diinginkan.

Pengantar Regex

pengolahan string dengan cara konvensional mengandalkan `split()`, `find()`, dan kondisi-kondisi manual menjadi semakin rumit ketika pola data yang dihadapi lebih kompleks. Regular expression, atau disingkat regex, hadir sebagai solusi yang lebih elegan dan ekspresif. Regex adalah sekumpulan karakter yang membentuk sebuah pola pencarian (pattern). Pola tersebut

kemudian dicocokkan dengan string lain untuk menemukan, mengekstrak, mengganti, atau menghapus bagian yang sesuai. Pada Python, fungsionalitas regex tersedia melalui modul standar `re` yang cukup di-import sebelum digunakan. Salah satu fungsi dasar dari modul `re` adalah `re.search()`, yang memeriksa apakah suatu pola ditemukan di dalam string.

Agar pencarian hanya mencocokkan baris yang diawali dengan pola tertentu, dapat ditambahkan karakter khusus `^` di awal pola. Dalam regex, `^` merupakan penanda bahwa pola harus ditemukan di awal baris. Penggunaan `re.search('^From:', line)` ekuivalen dengan pemanggilan `line.startswith('From:')`, namun regex memberikan fleksibilitas yang jauh lebih besar untuk pola-pola yang lebih rumit.

Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Library Python

Meta character adalah karakter-karakter dengan makna khusus dalam regex yang tidak diperlakukan sebagai karakter literal. Tabel di bawah ini merangkum meta character yang paling sering digunakan:

Karakter	Makna	Contoh Pola	Keterangan
[]	Himpunan karakter	[a-zA-Z]	Satu karakter antara a–z atau A–Z
.	Karakter apa saja kecuali newline	say.ng	Cocok dengan "sayang", "sabung", dll.
^	Diawali dengan	^From	String harus dimulai dengan "From"
\$	Diakhiri dengan	akhir\$	String harus berakhir dengan "akhir"
*	0 hingga tak terhingga	\d*	Nol atau lebih digit
?	0 atau 1 (opsional)	\d?	Boleh ada satu digit, boleh tidak
+	1 hingga tak terhingga	\d+	Minimal satu digit
{n}	Tepat n karakter	\d{4}	Tepat empat digit
()	Pengelompokan	(ab)+	Satu atau lebih pengulangan "ab"
	Atau (alternatif)	kucing anjing	Cocok dengan "kucing" atau "anjing"

Escaped Character (Karakter Khusus)

Selain meta character, regex juga mengenal escaped character — karakter yang diawali dengan tanda backslash \ dan memiliki arti tertentu. Escaped character ini sangat berguna untuk mencocokkan kategori karakter tertentu tanpa perlu mendefinisikannya satu per satu:

Escaped Char	Makna
\d	Digit angka (0–9)
\D	Bukan digit
\s	Whitespace (spasi, tab, newline)
\S	Bukan whitespace
\w	Karakter word (a–z, A–Z, 0–9, dan _)
\W	Bukan karakter word
\b	Batas kata (awal atau akhir sebuah kata)
\A	Awal dari keseluruhan string
\Z	Akhir dari keseluruhan string

Himpunan Karakter dengan []

Tanda kurung siku [] digunakan untuk mendefinisikan kelas karakter yang dapat dicocokkan. Beberapa aturan penting di dalamnya:

Pola	Keterangan
[abc]	Satu karakter: a, b, atau c
[a-c]	Satu karakter dalam rentang a hingga c
[^bmx]	Satu karakter yang BUKAN b, m, atau x
[0-9]	Satu karakter digit 0 hingga 9
[0-2][1-3]	Dua karakter: digit pertama 0–2, digit kedua 1–3

Pola	Keterangan
[a-zA-Z]	Satu huruf, huruf kecil maupun huruf besar

Empat Fungsi Utama Regex di Python

Modul re pada Python menyediakan empat fungsi utama yang masing-masing memiliki kegunaan berbeda:

Fungsi	Kegunaan
re.findall(pola, string)	Mengembalikan semua kemunculan pola sebagai list string
re.search(pola, string)	Mengembalikan objek Match untuk kemunculan pertama pola
re.split(pola, string)	Memecah string menjadi list berdasarkan pola sebagai pemisah
re.sub(pola, pengganti, string)	Mengganti semua kemunculan pola dengan string pengganti

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github: https://github.com/gathasja/71251218_Agatha.git

SOAL 1

```
1 def cekAnagram(kata1, kata2):
2     kata1 = kata1.replace(" ", "").lower()
3     kata2 = kata2.replace(" ", "").lower()
4     return sorted(kata1) == sorted(kata2)
5
6 kata1 = input("kata pertama: ")
7 kata2 = input("kata kedua: ")
8
9 if cekAnagram(kata1, kata2):
10     print(f"{kata1} dan {kata2} adalah anagram")
11 else:
12     print(f"{kata1} dan {kata2} bukan anagram")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> python -u "d:\KUL
ggu09\soal01.py"
kata pertama: apakabar
kata kedua: kabaraap
apakabar dan kabaraap adalah anagram
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> python -u "d:\KUL
ggu09\soal01.py"
kata pertama: anjing
kata kedua: jingga
anjing dan jingga bukan anagram
```

Penjelasan perlangkah soal01:

1. `def cekAnagram(kata1, kata2):` -> Mendefinisikan fungsi cekAnagram dengan 2 parameter.
2. `kata1 = kata1.replace(" ", "").lower()`
`kata2 = kata2.replace(" ", "").lower()` -> Hapus spasi dalam kata1 dan kata2 lalu ubah semua huruf menjadi kecil.
3. `return sorted(kata1) == sorted(kata2)` -> Urutkan huruf kedua kata, bandingkan. Jika sama maka True (anagram), jika beda maka False
4. `kata1 = input("kata pertama: ")`

kata2 = input("kata kedua: ") -> Meminta input kata dari pengguna

5. `if cekAnagram(kata1, kata2):` -> Memanggil fungsi `cekAnagram` untuk mengecek apakah kedua kata anagram
6. `print(f"{kata1} dan {kata2} adalah anagram")` -> Jika True, cetak pesan "adalah anagram"
7. `else:`
`print(f"{kata1} dan {kata2} bukan anagram")` -> Jika kondisi False (bukan anagram),
Cetak pesan "bukan anagram"

SOAL 2

```
1 def frekuensi(kalimat, kata):
2     kalimat = kalimat.lower()
3     kata = kata.lower()
4     kata_kata = kalimat.split()
5
6     jumlah = 0
7     for kat in kata_kata:
8         kata = kata.strip(".,!/:()\"'")
9         if kat == kata:
10             jumlah += 1
11
12     return jumlah
13
14 kalimat = input("masukan kalimat: ")
15 kata = input("masukan kata yang dicari: ")
16 hasil = frekuensi(kalimat, kata)
17
18 print(f"output: '{kata}' ada {hasil} buah")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAIPro> python -u
masukan kalimat: makan harus dua kali sehari. kalau cuma
masukan kata yang dicari: makan
output: 'makan' ada 2 buah
```

Penjelasan perlangkah soal02:

1. `def frekuensi(kalimat, kata):` -> Mendefinisikan fungsi frekuensi dengan 2 parameter
2. `kalimat = kalimat.lower()` -> Mengubah seluruh kalimat menjadi huruf kecil (case insensitive)
`kata = kata.lower()` -> Mengubah kata yang dicari menjadi huruf kecil
`kata_kata = kalimat.split()` -> Memisahkan kalimat menjadi list kata (dipisah spasi)
3. `jumlah = 0` -> Inisialisasi counter jumlah kemunculan = 0
4. `for kat in kata_kata:` -> Perulangan untuk setiap kata dalam list kata_kata
5. `kata = kata.strip(".,!/:()\"'")` -> menghapus tanda baca dari kata (harusnya dari kat)
6. `if kat == kata:` -> Membandingkan kata dalam kalimat dengan kata yang dicari
7. `jumlah += 1` -> Jika sama, tambah counter jumlah
8. `return jumlah` -> Mengembalikan total jumlah kemunculan.
9. `kalimat = input("masukan kalimat: ")`

kata = input("masukan kata yang dicari: ") -> Meminta input dari pengguna

10. hasil = frekuensi(kalimat, kata) -> Memanggil fungsi dan menyimpan hasil.

11. print(f"output: '{kata}' ada {hasil} buah") -> menampilkan hasil.

SOAL 3

```
1 def bersihkanSpasi(teks):
2     kata_kata = teks.split()
3     return ' '.join(kata_kata)
4
5 inputString = input("Masukkan string: ")
6 outputString = bersihkanSpasi(inputString)
7
8 print(f"String asli: {inputString}")
9 print(f"String bersih: {outputString}")
10
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> pyth
Masukkan string: hai      semua
String asli: hai      semua
String bersih: hai semua
```

Penjelasan perlangkah soal03:

1. `def bersihkanSpasi(teks):` -> Mendefinisikan fungsi bersihkanSpasi dengan 1 parameter teks.
2. `kata_kata = teks.split()` -> Memisahkan string menjadi list kata berdasarkan spasi. Spasi berlebih otomatis hilang.
3. `return ' '.join(kata_kata)` -> Menggabungkan list kata menjadi string dengan satu spasi sebagai pemisah.
4. `inputString = input("Masukkan string: ")` -> Meminta input string dari pengguna.
5. `outputString = bersihkanSpasi(inputString)` -> Memanggil fungsi untuk membersihkan spasi.
6. `print(f"String asli: {inputString}")` -> Menampilkan string sebelum dibersihkan.
7. `print(f"String bersih: {outputString}")` -> Menampilkan string setelah dibersihkan.

SOAL 4

```
1 def pendekPanjang(kalimat):
2     kata_kata = kalimat.split()
3     if not kata_kata:
4         return None, None
5
6     kata_terpendek = kata_kata[0]
7     kata_terpanjang = kata_kata[0]
8
9     for kata in kata_kata:
10        kata_bersih = kata.strip(".,!?:;()\\"")
11
12        if len(kata_bersih) < len(kata_terpendek):
13            kata_terpendek = kata_bersih
14        if len(kata_bersih) > len(kata_terpanjang):
15            kata_terpanjang = kata_bersih
16
17    return kata_terpendek, kata_terpanjang
18
19 kalimat = input("Masukkan kalimat: ")
20 terpendek, terpanjang = pendekPanjang(kalimat)
21
22 print(f"Kalimat: {kalimat}")
23 print(f"Kata TERPENDEK : {terpendek} ({len(terpendek)} huruf)")
24 print(f"Kata TERPANJANG: {terpanjang} ({len(terpanjang)} huruf)")
25
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Masukkan kalimat: hari ini adalah hari sabtu
Kalimat: hari ini adalah hari sabtu
Kata TERPENDEK : ini (3 huruf)
Kata TERPANJANG: adalah (6 huruf)
```

Penjelasan perlangkah soal04:

1. `def pendekPanjang(kalimat):` -> Mendefinisikan fungsi pendekPanjang dengan parameter kalimat.
2. `kata_kata = kalimat.split()` -> Memisahkan kalimat menjadi list kata (berdasarkan spasi).
3. `if not kata_kata:`
`return None, None` -> Jika list kosong (kalimat tidak ada isinya), kembalikan None, None (tidak ada kata).
4. `kata_terpendek = kata_kata[0]`
`kata_terpanjang = kata_kata[0]` -> Inisialisasi kata terpendek dan terpanjang dengan kata pertama.
5. `for kata in kata_kata:` -> Perulangan untuk setiap kata dalam list.
6. `kata_bersih = kata.strip(".,!?:;()\\"")` -> Menghapus tanda baca dari awal/akhir kata.
7. `if len(kata_bersih) < len(kata_terpendek):`

kata_terpendek = kata_bersih -> Jika panjang kata bersih < kata terpendek saat ini,
update kata terpendek.

8. `if len(kata_bersih) > len(kata_terpanjang):`

kata_terpanjang = kata_bersih -> Jika panjang kata bersih > kata terpanjang saat ini,
update kata terpanjang.

9. `return kata_terpendek, kata_terpanjang` -> Mengembalikan kedua nilai.

10. `kalimat = input("Masukkan kalimat: ")` -> Meminta input dari pengguna.

11. `terpendek, terpanjang = pendekPanjang(kalimat)` -> Memanggil fungsi dan menyimpan hasil.

12. `print(f"Kalimat: {kalimat}")` -> Menampilkan kalimat asli.

13. `print(f"Kata TERPENDEK : {terpendek} ({len(terpendek)} huruf)")` -> Menampilkan kata terpendek + panjangnya.

14. `print(f"Kata TERPANJANG: {terpanjang} ({len(terpanjang)} huruf)")` -> Menampilkan kata terpanjang + panjangnya.

SOAL 5

```
1 import re
2 from datetime import datetime
3
4 def ekstrak_tanggal(teks):
5     pattern = r'\d{4}-\d{2}-\d{2}'
6     tanggal_ditemukan = re.findall(pattern, teks)
7
8     return tanggal_ditemukan
9
10 def konversi_tanggal(tanggal_str):
11     tahun, bulan, hari = tanggal_str.split('-')
12     return f"{hari}-{bulan}-{tahun}"
13
14 def hitung_selisih_hari(tanggal_str):
15     tanggal = datetime.strptime(tanggal_str, '%Y-%m-%d')
16     sekarang = datetime.now()
17
18     selisih = sekarang - tanggal
19     return selisih.days
20
21 def main():
22     teks = input("Masukkan teks: ")
23     tanggal_list = ekstrak_tanggal(teks)
24
25     if not tanggal_list:
26         print("Tidak ditemukan tanggal dalam format YYYY-MM-DD")
27     else:
28         for tanggal_str in tanggal_list:
29             tanggal_baru = konversi_tanggal(tanggal_str)
30             selisih = hitung_selisih_hari(tanggal_str)
31
32             print(f"{tanggal_str} 00:00:00 selisih {selisih} hari")
33             print(f"Format baru: {tanggal_baru}\n")
34
35 if __name__ == "__main__":
36     main()
```

py"

Masukkan teks: Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawannasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 1783-06-08) dan KiHajar Dewant
ara (1889-05-02)

1945-08-17 00:00:00 selisih 29465 hari
Format baru: 17-08-1945

1785-11-11 00:00:00 selisih 87817 hari
Format baru: 11-11-1785

1783-06-08 00:00:00 selisih 88704 hari
Format baru: 08-06-1783

1889-05-02 00:00:00 selisih 50025 hari
Format baru: 02-05-1889

Penjelasan perlangkah soal05:

1. **import re** -> Mengimpor modul regex untuk pencarian pola teks.
2. **from datetime import datetime** -> Mengimpor class datetime untuk manipulasi tanggal.
3. **def ekstrak_tanggal(teks):** -> Mendefinisikan fungsi dengan parameter teks.
4. **pattern = r'\d{4}-\d{2}-\d{2}'** -> Pola regex: 4 digit - 2 digit - 2 digit (YYYY-MM-DD).
5. **tanggal_ditemukan = re.findall(pattern, teks)** -> Mencari semua pola tanggal dalam teks.
6. **return tanggal_ditemukan** -> Mengembalikan list tanggal yang ditemukan.
7. **def konversi_tanggal(tanggal_str):** -> Mendefinisikan fungsi dengan parameter string tanggal.
8. **tahun, bulan, hari = tanggal_str.split('-')** -> Memisahkan string dengan pemisah - menjadi 3 bagian.
9. **return f"{hari}-{bulan}-{tahun}"** -> Mengembalikan format DD-MM-YYYY.
10. **def hitung_selisih_hari(tanggal_str):** -> Mendefinisikan fungsi untuk menghitung selisih.

11. `tanggal = datetime.strptime(tanggal_str, '%Y-%m-%d')` -> Mengubah string menjadi objek datetime.
12. `sekarang = datetime.now()` -> Mendapatkan tanggal dan waktu saat ini.
13. `selisih = sekarang - tanggal` -> Menghitung selisih (hasilnya objek timedelta).
14. `return selisih.days` -> Mengembalikan selisih dalam hari.
15. `def main():` -> Fungsi utama program.
16. `teks = input("Masukkan teks: ")` -> Meminta input teks dari pengguna.
17. `tanggal_list = ekstrak_tanggal(teks)` -> Memanggil fungsi untuk mencari semua tanggal.
18. `if not tanggal_list:`
 `print("Tidak ditemukan tanggal dalam format YYYY-MM-DD")` -> Jika tidak ada tanggal yang ditemukan, tampilkan pesan.
19. `else:`
 `for tanggal_str in tanggal_list:` -> Jika ada tanggal, Loop untuk setiap tanggal.
20. `tanggal_baru = konversi_tanggal(tanggal_str)` -> Konversi ke format DD-MM-YYYY.
21. `selisih = hitung_selisih_hari(tanggal_str)` -> Hitung selisih hari.
22. `print(f"{tanggal_str} 00:00:00 selisih {selisih} hari")` -> tampilkan hasil.
23. `print(f"Format baru: {tanggal_baru}\n")` -> Tampilkan format baru + baris kosong.
24. `if __name__ == "__main__":` -> Mengecek apakah file dijalankan langsung (bukan diimport).
25. `main()` -> memanggil fungsi main().

SOAL 6

```
1 import re
2 import random
3 import string
4
5 def ekstrak_email(teks):
6     pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
7     emails = re.findall(pattern, teks)
8     return emails
9
10 def ambil_username(email):
11     return email.split('@')[0]
12
13 def generate_password(panjang=8):
14     karakter = string.ascii_letters + string.digits
15     password = ''.join(random.choice(karakter) for _ in range(panjang))
16     return password
17
18 def main():
19     teks = input("Masukkan teks: ")
20     emails = ekstrak_email(teks)
21
22     if not emails:
23         print("Tidak ditemukan alamat email dalam teks")
24     else:
25         for email in emails:
26             username = ambil_username(email)
27             password = generate_password(8)
28             print(f"{email} username: {username} , password: {password}")
29
30 if __name__ == "__main__":
31     main()
```

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> python -u "d:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro\PrakAlPro-71251218\minggu09\soal06.py"
Masukkan teks: Berikut adalah daftar email dan nama pengguna dari mailing list:anton@mail.com dimiliki oleh antoniusbudi@gmail.co.id dimiliki oleh budi anwarislamet@getnada.com dimiliki oleh slamet slumutatahari@tokopedia.com dimiliki oleh toko matahari
anton@mail.com username: anton , password: EALk4fin
antoniusbudi@gmail.co.id username: antoniusbudi , password: BPUUJP1m
anwarislamet@getnada.com username: anwarislamet , password: eh0GgbZ0
slumutatahari@tokopedia.com username: slumutatahari , password: tBVvh3ws

Penjelasan perlangkah soal06:

1. **import re** -> Modul regex untuk mencari pola email.
2. **import random** -> Modul random untuk memilih karakter acak.
3. **import string** -> Modul string untuk akses karakter (huruf, angka).
4. **def ekstrak_email(teks):** -> Fungsi untuk mencari email dalam teks.
5. **pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'** -> Pola regex untuk mendeteksi alamat email.
6. **emails = re.findall(pattern, teks)** -> Mencari semua email yang cocok.
7. **return emails** -> Mengembalikan list email.
8. **def ambil_username(email):** -> Fungsi untuk mengambil username dari email.
9. **return email.split('@')[0]** -> Pisahkan dengan @, ambil bagian pertama.
10. **def generate_password(panjang=8):** -> Fungsi membuat password random (default 8 karakter).

11. `karakter = string.ascii_letters + string.digits` -> Gabungan huruf besar + huruf kecil + angka.
12. `password = "".join(random.choice(karakter) for _ in range(panjang))` -> Pilih karakter acak sebanyak panjang, lalu gabung.
13. `return password` -> mengembalikan password.
14. `def main():` -> Fungsi utama program.
15. `teks = input("Masukkan teks: ")` -> meminta input dari pengguna.
16. `emails = ekstrak_email(teks)` -> Mencari semua email dalam teks.
17. `if not emails:`
 `print("Tidak ditemukan alamat email dalam teks")` -> Jika tidak ada email ditemukan, tampilkan pesan.
18. `else:`
 `for email in emails:` -> Jika ada email, loop untuk setiap email.
19. `username = ambil_username(email)` -> ambil username.
20. `password = generate_password(8)` -> Buat password random 8 karakter.
21. `print(f"{email} username: {username}, password: {password}")` -> tampilkan hasil.
22. `if __name__ == "__main__":` -> Mengecek apakah file dijalankan langsung (bukan diimport).
23. `main()` -> memanggil fungsi `main()`.